TECHNICAL BRIEFING

Agent Blast Radius and the Aksu Index

A complete account, from first principles to advanced, of the tool that measures what an Agentforce agent can actually reach — without ever invoking the agent: the need it came from, how it was built layer by layer, why each module exists, how each claim is proven, and answers to the hard questions you will be asked.

Aksu Software · 31 July 2026 · Every figure here is a real org measurement, a test run, or a named source. Where something was not measured, this document says so.

---

1. In one sentence

> **It statically measures whether the code behind an Agentforce agent can reach *more* than the user running that agent is permitted to see — and if so, which fields, with what proof.**

Three words: **static** (the agent is never invoked), **user-scoped** (not "is this code safe" but "what can this agent reach *as this user*"), **evidence-based** (every finding states how well it is proven).

2. The need it came from

Everyone says: "*Our agent only accesses the topics we defined.*" That sentence rests on **configuration** — the topic and action list in Agent Builder.

> **Configuration is a declaration of intent. Code is reality.**
> The gap has a name: the **permission gap**. The mechanism has an older one: the **confused deputy** — known since the 1980s, the delegate becoming more powerful than the principal it acts for.

Not a new problem. **What is new** is that the gap now has AI agents wandering through it. And it is not theory: SailPoint 2025 (Dimensional Research, 353 enterprise security professionals) reports **80%** of organizations saw unintended agent actions; the Cloud Security Alliance (April 2026) found **more than half** experienced outright agent scope violations.

Why was nobody measuring it?

Because it requires knowing three things **at once**, and they live in three worlds:

The knowledge	Where it lives	Who looks
Which actions the agent invokes	Agentforce metadata	Admin / AI team

What those actions' code reaches	Apex / Flow source	Developers
What the user may see	Profile + permission sets	Security / IAM

Nobody joins the three, because they belong to different teams. **That join is why this tool exists.**

3. Fundamentals (from zero)

3.1 Anatomy of an Agentforce agent

```
Agent  ──▶  Topic  ──▶  Action  ──▶  Apex class / Flow / Prompt template
```

Layer	What it is	Who defines it
Agent	The entity that talks to the customer; carries topics	AI team
Topic	A subject grouping: "invoice questions"	AI team
Action	A concrete task: "fetch the invoice"	Developer + AI team
Target	**Real code:** an Apex class, a Flow, a prompt template	Developer

The critical point: **the first three layers are visible in the configuration screen; the fourth is not.** Review usually stops at the third.

3.2 Apex and SOQL

```
public with sharing class InvoiceService {
    public Invoice__c fetch(Id invoiceId) {
        return [SELECT Id, Amount__c, Customer_IBAN__c
                FROM Invoice__c WHERE Id = :invoiceId];
    }
}
```

This one file carries **three** things the tool reads: the `sharing` declaration, the objects and fields the query reaches, and the **apiVersion** in the paired `.cls-meta.xml`.

3.3 Two separate security axes — the tool's most critical distinction

Axis	What it controls	Example question	In code
Record visibility sharing	Which *rows* you can see	"Can I see Ali's customers?"	`with/without sharing`
Object & field security CRUD + FLS	Which *objects and fields* you can access	"Can I read the IBAN field?"	apiVersion / mode clause

The classic trap — and it was walked into once

Consider a class at v58 marked `with sharing`. It:

- ✅ **honours** record sharing rules (thanks to the keyword)
- ❌ but **bypasses FLS entirely** (because v58 → system mode)

So it can read a field the user has **no permission on at all**. To claim "bounded by the running user", **both** axes must be bounded. This mistake was made once during development and **a live org run caught it** — the unit tests had not.

3.4 OWD: the default for record visibility

OWD	Meaning	For the permission gap
`Public Read/Write`	Everyone sees and edits	The record axis is already open — **bypassing sharing changes nothing**
`Public Read Only`	Everyone sees; owner edits	Same for reads
`Private`	Owner and hierarchy only	**Critical** — bypassing sharing is a real expansion

That is why **PS501** fires only on *system mode + Private OWD*. Flagging `without sharing` on a Public object would be noise — and **noise buries the real finding**.

3.5 User permissions: Profile, Permission Set, Permission Set Group

Structure	What it is	What the tool does
Profile	**Exactly one** per user; base permissions	Reads it
Permission Set	Additive package; many can be assigned	Reads all and **unions** them
Permission Set Group	A package of permission sets	Reads the platform's **computed aggregate**
Muting	**Takes back** a permission inside a group	Applied automatically via the aggregate

Why read the aggregate instead of computing from components?

Because **muting** makes the two differ. Measured in E9: a component still says "can read" while the group's computed aggregate has **no row at all** — muting applied. Had we summed the components we would believe the user is **more privileged** than they are, and **miss a genuine gap**.

This is an "error in the wrong direction": believing the user over-privileged makes the gap look **smaller** — an **optimistic** error, the most dangerous kind.

3.6 API version — the most critical, least known concept

```
InvoiceService.cls            ← the code itself
InvoiceService.cls-meta.xml   ← <apiVersion>58.0</apiVersion>   ⚠ THIS decides behaviour
```

> **The rule: v67 and above → user mode by default. v66 and below → system mode by default.**
>
> **Byte-identical source** runs bounded in one case and unbounded in the other, because of a number in a metadata file.

Measured in a real org (**E2**):

The same `without sharing` source file	Records returned
Deployed as API v58	**5**
Deployed as API v67	**0**

Same user, same data, same query. Also measured in **E2b**: at v67 a read of an FLS-hidden field does not return quietly empty — it **throws** ("No such column"). The platform blocks; it does not silently strip.

3.7 What "system mode" and "user mode" mean

Mode	Behaviour	When it applies
System mode	Code runs with **no** regard for user permissions	v66-and-below default, or `WITH SYSTEM_MODE`
User mode	Code is bounded by the running user's permissions	v67+ default, or `WITH USER_MODE`

System mode is not a bug — it is a deliberate Salesforce feature (background jobs, integrations need it). **The problem is system-mode code being wired behind an AI agent.**

3.8 What "running user" means for an agent

An agent is not a person, but every Salesforce operation runs in a user context — usually a service/integration user. **That is exactly what the tool measures:** "what does this agent reach as this identity?"

> The tool **separates two identity models and never blurs them in the report**:
>
> - `--running-user` → **a real person**: profile + all permission sets + groups
> - `--permission-set` → **a hypothetical model**: someone holding only that one set. No profile, no other sets. The report labels it "hypothetical grant model" — it is **not a person** and is never presented as one.

4. Anatomy of the problem: the chain

4.1 A real example in six steps

```
Step 1  Agent            Revenue Assistant
Step 2    └▶ Topic       "Invoice questions"        ← visible in configuration ✅
Step 3       └▶ Action GetInvoiceDetail             ← visible in configuration ✅
         ═══════════════════════════════════════════════════════════
                   THE CONFIGURATION REVIEW STOPS HERE
         ═══════════════════════════════════════════════════════════
Step 4             └▶ Apex  InvoiceService   (v58)
Step 5                └▶ Apex  BillingHelper (v58, without sharing)
Step 6                   └▶ SELECT ... Customer_IBAN__c   ← user has NO permission
```

4.2 Why each step is invisible to the previous reviewer

Step	Who looks	Why they miss the next one
1–3	AI team / admin in Agent Builder	The screen shows the action's **name**, not the code behind it
4	Developer in code review	`InvoiceService` has existed for years; **never associated with an agent**
5	Nobody	A helper class called from dozens of places, one of which is the agent
6	Nobody	The query line looks ordinary — **the danger is in the mode, not the field**

4.3 The chain widens in three different ways

Path	Mechanism	Rule
Record axis	`without sharing` + Private OWD → other people's records	PS501
Field axis	System mode → a field the user has no FLS on	PS502 / PS506
Write axis	System-mode DML → an object the user cannot write	PS503

4.4 A second example: the trigger cascade (subtler)

```
Action  →  Apex (v67, user mode ✅)  →  insert Order__c
                                              │
                                              ▼
                                       OrderTrigger (v52) ⚠
                                              │
                                              ▼
                                  WRITES to Audit_Log__c (user cannot write it)
```

Why is this subtle, and why does E6 matter?

The calling code is **v67 and in user mode** — it looks clean. But the trigger it fires runs at **its own apiVersion** (experiment **E6**). If the trigger is v52, its DML is in system mode and can write where the user cannot.

So **moving the agent's action to v67 is not enough**; the versions of the triggers it fires count too. The tool reports this as PS509, and gives **ERROR only when the trigger's own body proves the DML** — a legacy trigger alone is WARN.

4.5 What the tool does at each step

Step	Module	Operation
1–3	`agent_metadata_loader.py`, `agent_analyzer.py`	Reads the agent bundle, resolves topic/action, finds the targets
4–5	`apex_introspect.py` (+ AST)	Parses the source, follows class references one level
6	`apex_introspect._resolve()`	**Resolves the mode** of each query — the precedence law
+	`authority_analyzer.py`	Reach × user permissions × org labels → findings

5. The precedence law — the heart of the tool

The whole tool reduces to one question: **"in which mode will this single SOQL query run?"** Implemented as `_resolve()` in `apex_introspect.py`.

5.1 The three tiers

Rank	Source	Example	Effect
1	Explicit operation clause	`WITH USER_MODE` `AccessLevel.USER_MODE`	**Beats everything.** Declaration and version both irrelevant
2	apiVersion default	v67 vs v58	≥67 → **both axes** in user mode
3	Class sharing declaration	`without sharing`	**Record axis only**, and only under system mode

5.2 The special case: `WITH SECURITY_ENFORCED`

Axis	Result
Field security (FLS)	**Enforced** → `True`
Record visibility	**Untouched** → falls through to the sharing declaration

So `without sharing` + `WITH SECURITY_ENFORCED` = fields safe, **record axis still open**. Judged on one axis alone it would read "clean".

5.3 The write axis is a separate function

DML has its own resolver, `_resolve_dml_fls()`. The reason: **the write axis cannot be graded with the read axis's evidence.**

> **This distinction prevented a real error**
>
> A benchmark case was labelled `experiment:E2` — but E2 only ever **read** (5 rows vs 0). It never wrote, so it could not speak for DML's default. **Borrowed evidence reads as measured and isn't.** The label was corrected.

5.4 Three-valued logic: True / False / None

Value	Meaning	In the report
True	The axis is enforced	No expansion on this axis
False	The axis is not enforced	Proven expansion → **ERROR**
None	**Undetermined**	Honest unknown → **PS504 / WARN**

5.5 Resolution table — real combinations

apiVersion	Declaration	Clause	Record	FLS	Comment
v58	`without sharing`	—	False	False	Widest case — both axes open
v58	`with sharing`	—	True	False	**The trap:** says "with sharing", FLS still open
v58	*no declaration*	—	None	False	Record axis undetermined → honest unknown
v67	`without sharing`	—	True	True	Version beats declaration — **measured in E2b**
v58	`without sharing`	`WITH USER_MODE`	True	True	The clause beats everything — **E3**
v58	`without sharing`	`SECURITY_ENFORCED`	False	True	Fields safe only; records open

5.6 Why we never "fix" this law from documentation

The law was **not read from documentation; it was measured in an org** (E1–E3). Over the project's life, a claim that "this law is wrong" arrived **three times**, each citing genuine Salesforce documentation. **All three times the live org contradicted the document.**

The claim (sourced)	The measurement
v67 `without sharing` must bypass records	**E2b** refuted it — it does not
v67 triggers always run in system mode (Summer '26 release notes)	**E13** refuted it on Summer '26 — BLOCKED, 0 rows
`stripInaccessible(UPDATABLE)` strips nothing on a read path (this one was OUR belief)	**The runtime oracle** refuted it — it strips / throws

The rule: answer a documentation-based claim about *behaviour* **with an experiment, never with an edit**. Documentation describes intent; the org describes behaviour — and the customer runs the latter. **The third row matters most: there, we were the ones who were wrong**, and a rule that raised ERROR was corrected.

6. Built layer by layer

0 Measure first, code second

Before a line of analysis was written, 13 experiments ran in real orgs. The reason: **an analyzer is a model of behaviour; if the model is wrong, everything built on it is wrong.**

Experiment	What it proved
E1	The escalation is real: system mode 5 records, user mode 0 — at both the CRUD and sharing layers
E2	Same source: v58 = 5, v67 = 0. Also: "no declaration" ≠ `without sharing`
E2b	At v67 FLS **throws** rather than silently stripping
E3	`WITH USER_MODE` beats the declaration
E4	Compliance labels are free to read — but must be queried per object and are **FLS-gated**
E5	Flow `runInMode` is declarative → analysable without Apex parsing
E6	A trigger's DML runs at the **trigger's own** apiVersion
E8	Permission set groups resolve through the platform-computed aggregate
E9	**Muting** is applied in the aggregate — component says "can read", aggregate has no row
E10	Controlled: `without sharing`=5, no declaration=0, `with sharing`=0
E11	Platform event publish: v58 **lands**, v67 **blocked**
E13	A v67 trigger **is** bounded by the running user — refuting the documentation claim

1 Read Apex, resolve the mode — `apex_introspect.py` (788 lines)

- Reads the `sharing` declaration from the class header
- Reads `apiVersion` from `.cls-meta.xml`
- Finds every `[SELECT ...]`, subqueries and mode clauses
- Parses SOSL `RETURNING` clauses
- Applies the precedence law via `_resolve()`
- **Never guesses** what it cannot resolve → `None` → PS504

2 Read the agent's configuration

> **Why its own layer?**
>
> The same Apex class is dangerous in one agent and harmless in another. The question is not "is this code bad" but **"does this agent reach this code"**. Without reachability a finding is meaningless — and noise buries the real finding.

3 Resolve the user's real permissions

`permission_resolver.py` is a **pure function**: a snapshot goes in, effective permissions come out. No org connection — which is why it is **testable without an org**, and most tests work that way.

4 Read the org's own labels

> **The critical philosophy: we do not decide** which field is "sensitive". We read the `ComplianceGroup` labels the org's administrators applied. That is why the tool is not GDPR-specific — CCPA, HIPAA or an internal policy use the identical mechanism.

5 Join and produce findings — `authority_analyzer.py` (540 lines)

6 Report — `report.py` + `report_html.py`

7 The real parser (AST)

> **Why both regex and AST? Isn't that a weakness?**
>
> No. **AST is the default**; regex is the fallback so the tool still runs without Node. Both feed the **same precedence core**; they differ only in *extracting* reach.
>
> Compared over 104 real classes: **0 overclaims, 0 misses, 0 noise**. The residual is 22/104 **DEGRADED** — regex honestly says `None` where the AST resolves. **It says less; it never says something wrong.**

8 Agent Script and the data→prompt chain

Uses Salesforce's **own** Agent Script parser. PS520/521/522 trace data → variable → prompt hop by hop. Searching the *compiled* agent artifact for that chain returned **zero** occurrences — it exists only in Agent Script source.

9 The proof surface · 10 The Aksu Index

See sections 10 and 9.

7. Module by module

Module	Lines	Responsibility, and *why separate*
`cli.py`	401	Entry point; sequences the flow. Separate so analysis does not depend on a command line
`apex_introspect.py`	788	**The precedence law** + regex extractor. The most critical file
`apex_ast.py` + `ast_extract.js`	79+763	Real parse-tree bridge. A Node subprocess, because Python has no Apex parser
`authority_analyzer.py`	540	**The join and the rule engine**
`permission_resolver.py`	117	Pure function. Kept pure so it is **testable without an org**
`snapshot_loader.py`	158	User → profile + sets + PSG aggregate
`org_loaders.py`	437	Live `sf` queries
`agent_metadata_loader.py`	154	Agent bundle metadata
`agent_analyzer.py`	292	Agent graph, delegation (PS515), opaque actions (PS507)
`flow_introspect.py`	135	Flow XML → `runInMode` (no Apex parsing needed, per E5)
`genai_prompt_introspect.py`	180	Prompt templates — **all versions**, inactive included (PS513)
`agentscript_loader.py`	115	Bridge to Salesforce's own parser
`prompt_flow_analyzer.py`	149	PS520/521/522
`report.py`	490	Deterministic markdown, fingerprint, `aksu_index()`
`report_html.py`	819	Themed HTML, circles, stakeholder summary
`org_health.py` / `org_census.py`	269/276	"Beyond this agent" — whole-org signals
`verify_deterministic.py`	60	Runs the CLI twice, compares sha256
`make_pdf.py`	127	HTML → PDF. **Not part of the analysis**; it cannot change a verdict

Dependencies: Python 3.12, **standard library only** (no third-party Python packages) · Node.js **optional** (AST + Agent Script; degrades to regex without it) · Salesforce CLI (`sf`) for live reads · Edge for PDF rendering only.

8. Rule catalogue (PS501–PS522)

Every severity is a claim about **proof**, not impact: **ERROR = proven**, **WARN = a real boundary we could not prove**.

Rule	Severity	Fires when
PS501	ERROR/WARN	Potential record-scope expansion (system mode + Private OWD)
PS502	ERROR/WARN	Field read in system mode; the user has no FLS
PS503	ERROR/WARN	System-mode DML on an object the user cannot write
PS504	WARN	**Honest unknown** — dynamic SOQL, unresolved reach
PS505	WARN	Labelled field reaches the model although the user IS allowed it
PS506	ERROR/WARN	**The headline:** labelled field, invisible to the user, reaching the model
PS507	WARN	Opaque / standard action
PS508	WARN	Delegation deeper than one level
PS509	ERROR/WARN	Trigger cascade (ERROR only when the trigger's own DML proves it)
PS510	ERROR/WARN	Flow running in system mode
PS511	INFO	Pre-v67 class inventory
PS512	ERROR/WARN	`stripInaccessible` result discarded / wrong AccessType
PS513	ERROR/WARN	Latent reach in an **inactive** prompt-template version
PS514	WARN	Async / event / callout hand-off
PS515	INFO/WARN	Agent-to-agent delegation (WARN when unresolved)
PS516	WARN	**Formula field** — its inputs cannot be resolved
PS520/521/522	INFO/WARN/**ERROR**	The data→prompt chain, traced hop by hop

Why doesn't PS516 say "leak"?

A formula's *inputs* cannot be resolved; the user's FLS on the formula may not bound what its value **carries** — the one channel even v67 does not close. But the **platform behaviour was never measured** (experiment E12 could not be deployed). So PS516 is not a leak claim but a statement of **our own resolution limit** — true whatever the platform does. **We do not upgrade it without a measurement**, because we made exactly that mistake once (§5.6).

9. The Aksu Index: four buckets

```
Aksu Index: 6 proven (1 regulated) · 0 unproven boundaries · 1 unresolved
```

Bucket	Name	Meaning
P	Proven	Escalation proven (ERROR). **The headline number**
C	Classified	A **subset of P** — the org's own labels. Never added; already inside
B	Unproven boundaries	A real boundary we could not prove (WARN). **Never** merged into P
U	Unresolved	Reach that could not be determined (PS504). Printed, never dropped

> **The specification rule:** quoting P alone while U > 0 is a **violation**, and it is enforced in code: **no API returns fewer than all four numbers**.
>
> And a zero refuses to flatter itself. The real line from the clean org:
>
> ```
> Aksu Index: 0 proven (0 regulated) · 0 unproven boundaries · 2 unresolved
> (0 proven with unresolved reach is NOT clean — unknown never becomes clean)
> ```

10. The proof surface: how we know

This is the most important section. The real question about a security tool is not "*what do you find*" but ***"how do you know what you found is right?"***

10.1 Why "224 tests pass" is NOT an accuracy claim

> A test suite proves **the code does what its author expected**. It does not prove the author's expectation was **right**. If the same mind wrote both, a green test measures **consistency**, not **correctness**.
>
> So the proof surface has **six layers**, each measuring what the previous one cannot.

10.2 Layer 1 — Unit tests (224 tests, 12 files)

What they measure: the code's internal contracts. What they do **not** measure: whether the precedence law is actually correct.

10.3 Layer 2 — The Agent Authority Benchmark (28 cases)

A hand-labelled corpus. Each case declares the **exact** set of rules it must produce: anything more is a false positive, anything missing is a false negative.

> **The trap this file avoids — *mirror or oracle?***
>
> Generating expected outcomes **from the precedence law** would test the analyzer against a re-implementation of itself — a **mirror**, not an oracle. So **every label is hand-written**, with a `truth` field naming **where the ground truth comes from**.

Live benchmark output (31 July 2026):

Rule	TP	FP	FN	Precision	Recall
PS501	8	0	0	100%	100%
PS502	1	0	0	100%	100%
PS503	2	0	0	100%	100%
PS504	2	0	0	100%	100%
PS506	7	0	0	100%	100%
PS512	2	0	0	100%	100%
PS514	5	0	0	100%	100%
28 cases · 28 passed · 0 failed					

Note: PS511 (inventory) and PS507/PS508 (opaque/chain markers) are **deliberately not graded** — they are inventory and unknown-markers, not escalation claims. Counting them would **artificially inflate** the score.

10.4 Label strength — the benchmark's real quality metric

100% precision/recall is **nearly worthless on its own**. The real question is where the labels come from, and the corpus states it per case:

truth label	Meaning	Strength
`experiment:E`*n*	**Measured** in a real org (Milestone 0)	**Strong**
`experiment:runtime`	Measured by the **runtime oracle** — the org decided; re-runnable on demand	**Strongest**
`sfge`	An independent engine agrees	Medium
`platform-doc`	Documented semantics — **not measured here**	Weak
`reasoned`	The author's reasoning — **shares a mind with the implementation**	**Weakest**

Today's distribution (live output):

Label	Count	Comment
experiment	**21**	Measured in a real org — strong
platform-doc	3	Documented, not measured here
reasoned	**4**	Proves consistency, not correctness

> **Use this when presenting:** "Of the 28 cases, **21 were measured in a real org**. That the remaining 4 prove only consistency is something **we say ourselves** — the number is printed on every run."

10.5 Layer 3 — Mutation testing: does the corpus have teeth?

A benchmark passing on day one proves nothing on its own: it may simply agree with the implementation it was written beside. So the analyzer is **broken on purpose**, one semantic at a time, and we check whether the benchmark **notices**.

#	Semantic broken	Result
1	Precedence: always system mode	caught
2	Precedence: always user mode	caught
3	Precedence: ignore apiVersion (treat all as v58)	caught
4	False-positive guard: flag Id/Name/audit fields too	caught
5	Sanitizer: ignore `stripInaccessible`	caught
6	Async: ignore EventBus/Queueable/callouts	caught
7	Reach: ignore SOSL entirely	caught
8	Writes: treat all DML as user mode	caught
Mutation score: 8/8		

An **ESCAPED** mutation is not an error but a **finding**: it names a blind spot in the corpus. Today none escape.

10.6 Layer 4 — The runtime oracle: the org is the referee

> **The idea:** the analyzer **predicts**; the org **judges**.
>
> For every corpus case carrying a `runtime` shape, the oracle:
>
> 1. deploys the case to the org **as real Apex**
> 2. creates its own **throwaway user and permission set** — object read granted, and **deliberately no field permission** on `Customer_IBAN__c`
> 3. runs the code **as that user**
> 4. asks the one question that settles it: **"does that field actually come back?"**
>
> The generated test **asserts the analyzer's own prediction**. So **a failing test is not a broken test — it is the org saying the analyzer is wrong.**

There is a negative control too: a field the user *is* entitled to (`Secret_Data__c`) is seeded with a real value, so a successful read cannot be confused with "a null that would pass either way". **Escalation = the data came back AND the user was not entitled to it** — both facts, never one.

> **The oracle caught a real error — and it was ours**
>
> PS512/PS506 claimed that `stripInaccessible(AccessType.UPDATABLE)` on a read path "strips nothing, so the escalation stays proven", and raised **ERROR** on it. The label was `platform-doc` — a **belief**, not a measurement.
>
> The oracle refuted it in-org on **both branches**: without object Edit the call **throws**; with it, the field **is stripped**. The rule was corrected.
>
> **Two lessons:** (1) **a false positive costs credibility exactly as a false clean does**; (2) **one measurement is not a rule** — probe every branch of the axis before generalising.

10.7 Layer 5 — Determinism and the fingerprint

Same org, same user, same tool version → **byte-identical report**. Proven live: two runs, same sha256. The fingerprint seals not just the inputs but **the tool itself**:

Field	What it binds
`analyzer`	A **sha256 of the rule and extractor source**
`parser`	The parser version
`coverage`	What the analysis identity **could see**
`backend`	AST or regex
`apiVersion`	**Per action** — v58 and v67 resolve the same code oppositely

> **Why is there NO separate `schema_version` constant?**

> A task specification asked for one and it was **deliberately not added**: the rule schema *is* `authority_analyzer.py`, whose hash the fingerprint already takes. A separate constant would be redundant **and hand-maintained** — someone changes a rule, forgets to bump it, and the constant **lies**. A hash cannot be forgotten; a constant can.

10.8 Layer 6 — The sfge differential (an independent engine)

A systematic comparison against Salesforce's own **Graph Engine** over the **19 org-adjudicated cases**. The rules compared:

sfge rule	Our counterpart	Axis
`ApexFlsViolation`	PS502 / PS503 / PS506	Field & CRUD
`DatabaseOperationsMustUseWithSharing`	PS501	Record

Two fairness measures were taken:

- Cases are wrapped as `@InvocableMethod`, because sfge analyses **from entry points only**. A plain method would produce **zero violations** and read as "sfge agrees" — a **vacuous win**. It is also how an agent really calls Apex, so the shape is honest, not a convenience.
- Each case is scored **only on the axis its own shape adjudicates**. Scoring an unrefereed column is the classic way a differential flatters whoever wrote it.

Tool	Contradicts the org
Salesforce Graph Engine	**7 / 19**
Agent Blast Radius	**0 / 19**

The 7 are not one thing — say which:

Kind	Count	Explanation
apiVersion blindness	4	v67 read ×2, v67 write, v67 record. sfge wants an explicit check and **gives no credit for secure-by-default**
SOSL	1	`ApexFlsViolation` never walks `RETURNING` → it **misses** an escape the org hands over (a false *negative*)
Sanitizer rows	2	**The weakest two.** We do not call them clean either — we say **WARN**. A severity-discipline difference, not sfge being broken

> **NEVER present this as "sfge is bad."** It is a general-purpose, deliberately conservative scanner answering *"is an FLS check present?"*, with no notion of a **running user** and no notion of a **compliance label**.
>
> **The supportable claim is the narrow one:** for *this* question — what can this agent reach as *this* user — a version-aware, user-scoped analysis is measurably more precise. Also, on **sfge's own binary scale** (any finding = an assertion) the score is **2/19**; stating both is honesty — publishing only the flattering one is selective reporting.

10.9 What no oracle can settle — a limit of the method

Some cases assert what the **analyzer must report**, not what the platform does. An org **cannot** have an opinion about them:

Case	Why the org cannot judge
`unknown-dynamic-soql` `unknown-sosl-without-returning`	PS504 says **"we do not know"**. An org cannot measure **the absence of our knowledge**
`async-queueable`, `async-callout`	PS514 says **"we flag a hand-off we do not follow"**. Same shape of claim
`field-untagged-escalates-ps502`	Its platform half is already measured; the half that distinguishes it is **our own labelling design**

Counting these as gaps would overstate what is missing, exactly as counting them as measured would overstate what is proven. The corpus records, in writing, which is which and why.

10.10 Layer 7 — Real orgs

Org	Role
HospitalOrg	Experiment lab (E1–E13) + a live agent
HanseWatt	Agent at v67 → **0 proven**, though the org is 83% legacy
TechnoStore	113 classes, **100% pre-v67** → the legacy demo, **6 proven**
Urla	A control org with no agent

House rule: after every rule change, **both demo orgs are run**. Unit tests once missed an early `return` that made a block unreachable — the live run caught it instantly.

11. The hard questions you will be asked

Grouped, each answer backed by a measurement. Do not memorise the answers — understand the **reasoning**; if the question arrives in another shape, the reasoning carries you.

A · POSITIONING AND COMPETITORS

> **Q1: Doesn't Salesforce's own Health Check / Optimizer do this?**
>
> No. Those examine **org-wide configuration** health — password policies, session settings, unused permissions. None asks *"what does this agent's code chain reach for this user?"* Theirs is **org-scoped**; ours is **agent- and user-scoped**.

> **Q2: Graph Engine already exists. What is different?**
>
> sfge asks *"is there an FLS check in the code?"* We ask *"can this user see this field, and does the code reach it?"* sfge has no concept of a **running user**, none of a **compliance label**, and **does not consider apiVersion** — so it warns about code the platform has already made safe at v67. Across 19 adjudicated cases the difference measured **7 to 0**. But sfge is not a bad tool; it answers a different question, and being conservative there is **a correct design choice**.

> **Q3: A consultant could do this by hand. Why a tool?**
>
> By hand — **once**. Then it is needed again on every deploy. Three differences: **(1)** reproducible (same input, byte-identical report), **(2)** runs in CI (`--fail-on ERROR`), **(3)** it **does not forget** — a human may skip step 5 of the chain; the tool does not. And a consultant bills by the hour; this measurement takes minutes.

> **Q4: Is this a security certificate?**
>
> **No, and we say so first.** It is an **agent-scoped security review accelerator** producing **evidence** for a DPIA or security review. Calling it a certificate would mean guaranteeing things **we never measured** — and §12 lists exactly those.

B · METHOD AND TECHNIQUE

> **Q5: Why not run the agent and observe it?**
>
> Three reasons. **(1) Coverage:** running shows only what happened in *that* conversation; we measure everything that *could* happen — a sample of conversations is never complete. **(2) Cost:** every invocation burns Flex Credits. **(3) Risk:** running an agent against a live org can produce side effects. Static analysis solves all three and is **reproducible**.

> **Q6: You use regex? Can that be serious analysis?**
>
> **The default is not regex — it is a real ANTLR parse tree**; regex is the fallback used only when Node is unavailable. Compared over 104 real classes: **0 overclaims, 0 misses, 0 noise**. The residual is 22/104 DEGRADED — regex **honestly says "I don't know"** where the AST resolves. And crucially both feed the **same precedence core**, so the **verdict** is issued in one place.

> **Q7: How do you resolve dynamic SOQL?**
>
> **We don't — and we say so.** `Database.query(...)` takes a string built at runtime; it cannot be known statically. PS504 fires: "unresolved". **Guessing would manufacture a silent false clean** — the one thing this product does not accept.

> **Q8: How many levels of the chain do you follow?**
>
> Class references are followed **one level** (including Queueable/Batch/@future — measured, not assumed). Anything deeper is flagged as a delegation chain via **PS508**. This is a limit and **it is in the report**: declaring what we do not know beats concealing it.

> **Q9: Can you see inside managed packages?**
>
> No — the source is not readable. This is a **disclosed limit** and it appears in the report. Knowledge/Data Cloud retrieval is likewise opaque today. If a field comes from inside a managed package, the tool marks it **unresolved**, **not clean**.

> **Q10: Why Python? Why not an Apex or AppExchange package?**
>
> Because the analysis must run **outside** the org: code running inside would sit **within** the very boundaries it measures — a ruler measuring itself. It also needs to run in CI on every deploy. Python 3.12 with **standard library only** was chosen so installation friction is near zero.

> **Q11: If we move to v67, the problem goes away — right?**
>
> Largely yes, and we **measured** it. Two caveats. **(1)** One org we measured is **83% legacy** and still measured **zero**, because the nine actions its agent invokes are v67. So modernise **the part the agent touches**, not the whole org — a far cheaper roadmap for the customer. **(2)** v67 does not close formula-field inputs (PS516), and **triggers run at their own versions** (E6).

C · TRUST AND EVIDENCE

> **Q12: Can this tool break something in my production org?**
>
> **No.** The **complete** list of what it runs against your org: `sf data query` (SOQL SELECT) and `sf project retrieve start` (metadata download). No insert/update/delete, no deploy, no Apex execution, no agent invocation. A SOQL SELECT **fires no triggers, locks no records, changes no data**.
>
> **And better still:** connect with a **read-only integration user** — then the guarantee is not "the tool does not write" but **"the tool cannot write"**. *The honest footprint:* SOQL consumes API allowance and `--include-counts` can be slow on very large objects. We do not hide that.

> **Q13: You claim "100% accuracy" — why should I believe it?**
>
> **Don't believe it, interrogate it — we did.** That figure holds **on our own 28-case corpus** and is **nearly worthless on its own**. The real question is where the labels come from: **21 were measured in a real org**, 3 rest on documentation, and **4 prove only consistency** — and that distribution is **printed on every run**. The corpus was also shown to have teeth by **mutation testing** (8/8), and the predictions were tested **with the org as referee**.

Q14: How exactly does the runtime oracle work?

It deploys the case to the org **as real Apex**, creates a **throwaway user and permission set** (object read granted, **deliberately no permission** on the target field), runs the code **as that user**, and asks one question: "*did the field come back?*" The generated test **asserts the analyzer's own prediction** — so **a red test is the org saying the analyzer is wrong**. There is also a **negative control**: a field the user genuinely may read is seeded with a value, so a pass cannot be a null that would have passed either way.

Q15: Have you ever been wrong?

Yes — and this is the most valuable answer. Our belief about `stripInaccessible(UPDATABLE)` was wrong and produced a rule that raised **ERROR**. The runtime oracle refuted **both branches** in-org and the rule was corrected. Twice more, well-founded questions arrived citing real Salesforce documentation, and **both times we ran an experiment rather than an edit**. **A false positive costs credibility exactly as a false clean does** — we learned that by paying for it.

Q16: If I run the report twice, do I get the same thing?

Yes, byte for byte — proven live (two runs, same sha256). The report carries a **fingerprint**: a sha256 of the rule/extractor source, the parser version, what the analysis identity could see, the backend, and **each class's own apiVersion**. A verdict is reproducible only **against the tool that produced it** — and the report **says so** rather than implying it. The live `COUNT()` lines are **outside** the fingerprint, and the report says that too.

Q17: Where does our data go?

Nowhere. The analysis runs on the local machine. No external service calls, no telemetry. What is read is **metadata and permission configuration** — not customer records and not conversation content.

D · NUMBERS AND REPORTING

Q18: How many records can it reach? Why no exact number?

Because record visibility **depends on sharing** and cannot be known statically. An org `COUNT()` is an **upper bound** — predicates and `LIMIT` are not resolved. The report says "*could reach up to N records*", never "*reaches N*". **A fabricated number is worse than no number.**

Q19: Why split ERROR and WARN? Isn't everything a risk?

For us severity is a claim about **proof**, not impact. ERROR = proven. WARN = a real boundary we could not prove. Without that split, a report equates the proven with the suspected — and **the first false positive costs the whole report its credibility**. Ranking by impact is the customer's job; ranking by proof is ours.

Q20: The Index came out 0. Is my org clean?

No — and the report says so in its own voice. If "unresolved" > 0 the line below reads: "*0 proven with unresolved reach is NOT clean — unknown never becomes clean.*" That is the specification's rule and it is **enforced in code**: no API returns fewer than four numbers. And because restriction rules are not modelled, the gap is a **lower bound**.

> **Q21: Why say "regulated" instead of GDPR?**
>
> Because **we do not decide** which field is regulated — we read the `ComplianceGroup` labels **the org's own administrators** applied. One org may tag for GDPR, another for CCPA or HIPAA; **the mechanism is identical**. Calling it "a GDPR tool" would describe a regime-agnostic feature as if it were one customer's use of it.

> **Q22: Can I suppress a finding? (CI noise)**
>
> **Not today — and it is a known gap**, raised by external review too. **A CI gate nobody can silence gets switched off**, so this is a real product risk. If it is built, the honest shape is known: an inline directive **with a mandatory reason**, and **unreasoned suppressions listed in the report**.

12. Honest limits

State these **before you are asked** — the cheapest credibility available.

Limit	Status and direction
Formula field inputs	Reported via PS516; platform behaviour **not measured**, not upgraded to a leak claim
Polymorphic lookups	A lookup with several targets is skipped — we cannot know which object a row points at, so it stays **unlabelled**. A coverage hole, and we say so
Restriction / scoping rules	Not modelled. **The direction is critical:** they *reduce* what the user sees, so omitting them makes us think the user is **more privileged** → the gap is a **LOWER BOUND**, not an upper one
Managed package internals	Opaque — marked unresolved
Record counts	Upper bound — predicates and LIMIT not resolved
Async subscribers	Publish is analysed; **Flow / process / external subscribers are not followed** (PS514 says exactly this)
Suppression / baseline	None — a known gap (Q22)
Chain depth	Class references one level; deeper is flagged via PS508

> **The closing line:** "This tool does not give you a perfect answer. It gives you a **measured** one — and it tells you, **in the same report**, what it could not measure. A silent 'clean' result is far more dangerous than an honest 'I don't know'."
